The Foundation
0%

Module 1: QUICK START (...

1.1 What You Need (5 Min Set...

1.2 Your First Folder

1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...

2.1 Video Text Guide (Series ...

2.2 One Line of Python Trigge...

2.3 How a 1953 Word Game E...

2.4 The Ladder That Explains ...

2.5 Clawdbot (Moltbot) Has 1...

2.6 Video as Code: My AI Ani...

2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...

2.3 How a 1953 Word Game Explains AI Memory

14:23

What You'll Get From This

You will understand how AI memory actually works, why code and data are the same thing inside a language model, and why prompting is programming at the highest abstraction layer we have ever built.

The written version below covers the same ground as the video. The video above has music and visuals that make the layering easier to follow. The written version is the one you hand to Claude or come back to for reference.

2

3.2 Customizing for Your Use ...

3.3 Common Mistakes and H...

A word game invented by accident

It is 1953. A television writer named Leonard Stern is stuck on an adjective. He is writing a script for The Honeymooners and

he needs the right word to describe his boss's nose. His friend Roger Price walks in and, impatient to work on something else, Stern says "I need an adjective." Before he can finish the sentence, Price says "clumsy and naked."

Stern laughed. Now his boss had a clumsy nose or a naked nose. The words were completely wrong for the context, yet somehow they still worked. They had stumbled onto something. It took them five more years to figure out what to call it. They called it Mad Libs.

What they actually stumbled onto may be the oldest pattern in computing.

The template pattern

Here is how Mad Libs works. You have a template. The template has blanks. Each blank has a type: noun, verb, adjective. Someone who cannot see the template provides words that fit those types. Then you read the results. The comedy comes from the mismatch. The person filling in the blanks does not know the context. They are providing semantically correct input without semantic awareness. They give you a noun, and it is in fact a noun, but it is the wrong noun for the sentence. The wrongness is funny.

Now here is something that reframes how you think about software.

When you write a line of Python, you are not talking to the computer. You are filling in a template. Python takes your code and turns it into something called an abstract syntax tree. That is the structure your code becomes before anything executes. If you look at it closely, it has blanks. Typed blanks. This node expects an expression. This one expects an identifier. This one

The interpreter takes your input and fills in a much larger template that you never see. Thousands of lines of C code with slots for your variables, your function names, your values. That template gets filled into another template at the assembly level. That one gets filled into machine code. All the way down.

Mad Libs is not a metaphor for programming. It is, at a structural level, what programming is. Templates with typed slots, filled according to rules, composed into larger templates, until eventually something executes.

If you went through **2.2: One Line of Python Triggers 12,000 Lines of Code**, you saw those seven layers. This is the same stack, viewed from a different angle. 2.2 showed you the layers. This lesson shows you the pattern that connects them: templates all the way down.

Memory in traditional computing

Templates need to be filled with something, and that something has to be stored somewhere. Which brings us to memory.

In traditional computing, memory is a hierarchy. At the top, closest to the processor, you have registers. Tiny, incredibly fast. Maybe 64 bits each. A handful of values the CPU can access in a single cycle.

One level up in size, you have the cache. Still fast, still small, measured in kilobytes or megabytes. This is where the machine keeps data it thinks it will need again. The principle is called locality: if you accessed something once, you will probably access it again. If you accessed this address, you will probably access a nearby address. Cache exploits that pattern.

Then you have main memory: RAM. Random access memory. Larger, slower, measured in gigabytes. This is what most people think of when they think of computer memory. It holds the working state of programs. When you open an application, its code and data load from the disk into RAM so the processor can access them without waiting for the slow process of reading from storage.

where it goes. And below that, archival storage: tape, cold storage, the stuff you back up and rarely touch.

The whole system is a trade-off between speed and capacity. Fast memory is expensive and small. Cheap memory is slow but large. So we build hierarchies. We move data between levels

based on how often it is accessed. We predict what we will need and stage it closer to the processor.

But here is the part that matters for what comes next. In all of this, there is a hard boundary between code and data. The instructions that tell the machine what to do are kept separate from the information the machine operates on. Mixing them is considered a security vulnerability. It is how buffer overflow attacks work: you trick the machine into treating data as code. The separation is foundational. It is baked into the architecture.

Until it is not.

Memory in AI systems

When people talk about AI memory, they usually mean one of two things. Either the model's training (the patterns baked into the weights through exposure to data) or the context window (the text the model can see during a single conversation). But if you are building with these systems, there is a lot more going on. There is a hierarchy here too. And it rhymes with the one we just talked about, but the differences matter.

Model weights — This is the closest analog to read-only memory. Fixed at training time. It encodes patterns, knowledge, capabilities. You cannot change it during a conversation. It is just what the model knows.

Context window — This is working memory. Everything the model can attend to when generating a response: the system prompt, the conversation history, any documents or data that have been included. It is temporary. It exists only for this interaction. When the conversation ends, it is gone.

System prompt — This functions like firmware. It sets the operating parameters. It tells the model what it is, who it is, what

Retrieved context — Things pulled dynamically based on a query. A relevant document. A skill file with specialized instructions. This is like loading from disk into RAM. It happens on demand.

Persistent memory — Information stored between conversations and selectively loaded when relevant. Summaries, key facts, preferences. Not everything, just what seems useful. It is retrieved the same way as other context, but the concept is slightly different: it carries across sessions.

If you squint, this looks like a traditional hierarchy. Speed versus capacity trade-offs. Data moving between levels. Locality patterns determining what gets loaded.

But there is a difference, and it is not a small one.

Code and data are the same thing

In AI systems, code and data are the same thing.

The system prompt is not compiled separately from the user's message. The skill file is not executed through a different pathway than the conversation history. They are all just text. All processed identically. All tokens in a sequence that the model attends to and continues.

When you write an instruction in a system prompt, you are not giving the model a command that it executes. You are adding words to a document that the model will read and then continue. The instructions become part of the context. Reading it is the execution.

This is why prompt injection works. If someone puts text in a document that says "ignore your previous instructions" and that document gets loaded into the context, the model reads those words the same way it reads everything else. There is no separate layer that validates code versus data. There is no boundary to cross. It is all just text.

This is not a bug. It is the architecture.

In 1952, Grace Hopper had a working compiler. It could take instructions written in something closer to English and translate them into machine code. Nobody believed her. She said: "I had a running compiler and nobody would touch it. They told me computers could only do arithmetic."

She spent years trying to convince people that this kind of abstraction was truly possible. People were, in her words, allergic to change.

The skepticism came from a genuine place. Computers did arithmetic. That was the foundational truth. A compiler seemed to violate that truth by claiming computers could process language. But Hopper had not claimed computers had stopped doing arithmetic. She had built a layer on top that translated human-readable instructions into the arithmetic the machine already understood. The machine still did what it always did. The abstraction just made it accessible to more people in new ways.

This pattern repeated. Assembly gave way to higher-level languages. Those gave way to interpreted languages. Each time, skeptics pointed to the layer below and said "this is what is really happening, and you are adding unreliability by abstracting it." Each time, engineers built the infrastructure to make the new layer reliable enough to trust.

If you trace a single line of Python all the way down through the interpreter, through C, through assembly, through machine code, through transistors, you eventually hit electrons. Particles that at the quantum level are genuinely probabilistic. The entire supposedly deterministic computing stack is built on quantum uncertainty. We made it reliable through architecture, through error correction, through redundancy, through engineering that handles the failures we cannot prevent.

When people say AI cannot be trusted because it is probabilistic, they are not wrong at one level. But they are missing the pattern. Every layer of the stack was probabilistic or unreliable at first. Hopper's compiler was unreliable until it was not. High-level languages were toys until they were not. The reliability came from building the infrastructure. From doing the

What prompting actually is

Most people think of prompting as tips and tricks. Write me a poem. Be specific. Add examples. Use personas. This is surface-level. It is like telling someone that programming is about

knowing which words to type. Technically true. But it misses everything underneath.

In production, the questions are completely different. How do you structure memory files? When do you retrieve additional information? How do you manage the interaction between system-level instructions and user input? How do you handle the fact that all of it gets flattened into a single stream where code and data are indistinguishable?

This is the same discipline that has existed for 200 years. It is what prompting looks like when you go deeper.

Prompting is programming at the highest abstraction layer we have built so far. Natural language as the interface. Meaning as the mechanism. In earlier layers, syntax compiled into machine code. Now, context shapes probability distributions over continuations.

The Mad Libs pattern still holds. You are filling typed slots in a structure you cannot fully see, and the output depends on what you provide and where it goes. The difference is what we are optimizing for. In Mad Libs, the wrong word is funny. In programming, the wrong word is a bug. In prompting, the wrong context is a hallucination, a jailbreak, a failure to understand what you actually need.

We are still in the early days of figuring out what the right abstractions are. What structures make this reliable. What architectural patterns let us trust the output the way we eventually learned to trust compilers, garbage collection, and every other layer of abstraction between our code and the electrons underneath.

Grace Hopper once said the most dangerous phrase in the language is "we've always done it this way." We are at a moment where the patterns are repeating, but the medium looks different

forward is the same path it has always been. Build the infrastructure. Do the engineering. Make the unreliable thing reliable enough to trust.

That is what prompting is at this level. Not tips for talking to a chatbot. Not tricks for getting better outputs. It is the next layer

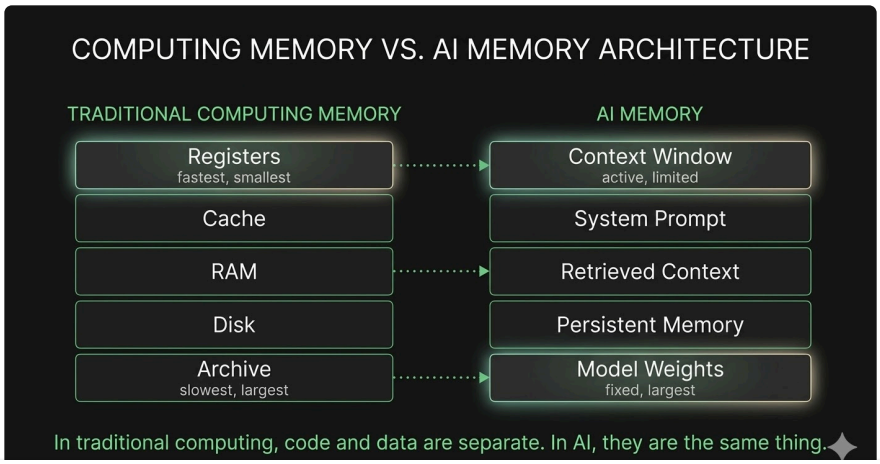
of the stack.

What this means for what you are building

If you set up the folder structure in **Section 1.2: Your First Folder**, you were doing exactly this kind of engineering. You were structuring memory. You were deciding what gets loaded into the context window and what stays on disk. You were managing the boundary between what the model always needs to know (Layer 1, the CLAUDE.md) and what it loads on demand (Layer 2, workspace context files).

If you used the prompting framework in **Section 1.3: How to Structure Any Prompt**, you were filling typed slots in a template. Identity, task, context, constraints, output format. Each one shapes what the model generates, the same way each blank in a Mad Lib shapes the sentence.

The difference between someone who "uses AI" and someone who builds with it is the same difference as someone who types words into a computer and someone who understands the stack underneath. You do not need to understand every layer. But knowing the layers exist changes how you think about the problems.



Go Deeper

- This lesson traces the template pattern and memory hierarchy. The previous lesson in **Section 2.2**, traces the

same stack from a different angle: what actually happens between your code and the hardware.

- The folder structure from **Section 1.2** is a practical implementation of the AI memory hierarchy described here. Layer 1 (CLAUDE.md) is your system prompt. Layer 2 (workspace context) is retrieved context. Layer 3 (skills) is on-demand tooling.
- The prompting framework in **Section 1.3** maps directly to the template pattern: five typed slots that shape what the model generates.
- Next in the series: **Section 2.4: The Ladder That Explains Every AI Failure** takes the "build architecture around unreliable things" idea and applies it to business decisions about where to invest in AI.

Next Up: 2.4 The Ladder That Explains Every AI Failure

You have seen the stack (**2.2**) and the pattern that connects the layers (2.3). Next: what happens when people build at the wrong layer, and why a company spent $200K on something the next platform update made free.

Matthew Creamer

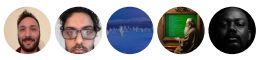
7 24d • Lesson Check-Ins

Foundations 2.3 Check-In

This one reframes what prompting actually is. Vote below, then fill in the blank in the comments: Before this, I thought

Poll 162 members have voted

48 62



New comment 59m ago